

项目简介

项目名称：健康膳食系统

主要功能：为用户提供每日饮食推荐、健康状况记录、营养素分析及智能问答等。系统基于 Web，包含前端交互、后端业务逻辑、数据库及推荐算法模块，涉及用户隐私数据（身高、体重、过敏原等），需兼顾功能性与合规性。团队规模 3 人，开发周期约 1 个月，采用敏捷开发模式。

软件过程定义来源

本过程的定义以 ISO/IEC 12207:2017 《系统与软件工程——软件生命周期过程》为核心框架，同时参考了 ISO/IEC 15504（SPICE）中过程评估模型对过程执行属性的要求，以及 ISO/IEC 33001 关于过程能力评估的概念，确保定义的过程不仅覆盖必要活动，还具备可评估和改进的基础。

软件过程定义

根据 ISO/IEC 12207 的过程组结构，结合项目实际，选取并定义了以下软件生命周期过程。每个过程均给出目的、主要活动，并保留了与标准框架的映射关系。

1. 协议过程组

获取与供应过程

目的：定义系统采购（如第三方 API）与交付成果的协议。

活动：撰写接口需求规格说明书，与外部营养数据 API 供应商签订服务协议，明确 SLA；与用户代表（健康管理师）签订验收标准备忘录。

2. 组织项目使能过程组

生命周期模型管理过程

目的：界定并维护项目生命周期模型。

活动：选定“Scrum+阶段插入”混合模型，定义 7 个迭代，在迭代 0 进行架构设计与数据集构建，迭代 1~6 开发业务功能，迭代 7 进行系统验收与上线。

基础设施管理过程

目的：建立并维护开发环境、工具链。

活动：配置版本控制仓库（Git）、CI/CD 流水线（GitHub Actions）、自动化测试框架、需求管理看板（Jira）。

质量保证过程

目的：确保过程与工作产品符合要求。

活动：定义检查单，每迭代末进行代码评审、架构评审；每周进行风险回顾；

3. 技术管理过程组

项目规划过程

目的：制定并维护项目计划。

活动：估算用户故事规模（故事点），制定发布计划与迭代计划；定义 DoR/DoD；每日站会

同步进度。

项目评估与控制过程

目的：监控项目绩效，采取纠正措施。

活动：燃尽图跟踪迭代进度，代码提交频率与缺陷密度分析；每周演示与回顾。

风险管理过程

目的：识别、分析并缓解项目风险。

活动：注册风险清单，重点监控“推荐算法准确性不足”、“用户隐私合规性”、“数据源不稳定”等风险；每迭代更新风险状态。

决策管理过程

目的：对重大问题做出结构化决策。

活动：若出现架构变更或需求分歧，由产品负责人、技术负责人共同评审，采用决策矩阵记录。

配置管理过程

目的：控制基线变更与产品版本。

活动：源代码、数据集、模型、配置文件均纳入版本管理，对数据集的标注版本和模型参数进行基线控制。

信息管理过程

目的：管理项目信息及用户健康数据的生命周期。

活动：定义数据分级与脱敏策略；构建数据流图，确保个人健康数据仅用于必要目的，符合隐私法规。

4. 技术过程组

业务或使命分析过程

目的：理解健康膳食业务问题与目标。

活动：与营养师、潜在用户进行访谈，定义业务目标量化指标（例：用户每日营养素达标率）。

利益相关方需求定义过程

目的：将利益相关方的期望转化为结构化需求。

活动：构建用户故事地图，区分用户（普通用户、管理员）需求，明确隐私约束与交互要求。

系统软件需求分析过程

目的：指定系统的技术需求。

活动：由于是完全软件系统，将系统需求与软件需求合并，输出系统需求规格说明书（SRS），包含功能需求、性能要求、安全要求及推荐算法准确率指标。

架构设计过程

目的：建立系统的顶层结构。

活动：选定前后端分离架构（Vue.js + Spring Boot），设计微服务雏形（用户服务、膳食推荐服务、问答服务、数据统计服务），使用 4+1 视图描述。

详细设计过程

目的：对各组件内部进行详细设计。

活动：设计 RESTful 接口、数据库逻辑模型、算法模块类图；定义推荐模型的训练策略与接口。

软件构建过程

目的：编写并单元测试代码。

活动：TDD 方式开发，单元测试覆盖率>70%；多 AI 辅助代码生成，但必须通过人工审查与测试。

软件集成过程

目的：组装软件单元，形成完整系统。

活动：CI 每日集成，冒烟测试自动运行；每迭代末进行功能集成测试。

软件合格性测试过程

目的：确认软件满足需求。

活动：执行系统测试用例，含功能、性能、安全测试；邀请真实用户进行可用性测试（UAT）。

系统合格性测试过程

目的：验证最终系统满足业务需求。

活动：因无独立硬件，将系统测试与验收测试合并，由产品负责人和营养师共同验收。

交付过程

目的：将系统部署到生产环境并向用户交付。

活动：灰度发布、监控上线后性能，撰写用户手册与常见问题。

软件维护过程

目的：修复缺陷、优化功能。

活动：建立维护窗口，收集用户反馈，按优先级排入下一迭代。

软件退役过程（按需激活）

目的：当系统不再使用时，安全下线并处理数据。

活动：预留数据导出与系统下线方案，满足用户数据可携带性要求。

5. 特殊新增过程

数据管理过程

目的：规范饮食推荐所需数据的采集、标注、清洗与存储。

活动：建立数据采集管道，设计标注规范，监控数据质量。

模型开发与训练过程

目的：开发智能推荐与问答模型。

活动：遵循 CRISP-DM 方法论，进行数据探索、特征工程、模型训练、评估（准确率、多样性），模型版本与代码一同管理。

剪裁原则

依据 ISO/IEC 12207 允许的剪裁规则，围绕“项目规模小、团队年轻、功能安全程度中等、技术栈现代、用户体验优先”的特性，采用“复用标准结构、合并冗余过程、弱化不适用过程、补充领域特有过程”的思路。

具体剪裁决策与理由

合并系统需求与软件需求：项目无独立硬件，系统边界清晰，避免重复文档，提高迭代速度。

弱化正式的获取过程：无外包开发，仅调用外部 API，将重点转移至接口协议与 SLA，省去招标、合同谈判等重型活动。

省略独立的质量审计或认证过程：团队小，将质量保证内化到每日工程活动中，参照 15504 能力等级 1~2 的要求执行，但不设立独立 QA 角色。

增加数据管理与模型开发过程：膳食推荐和智能问答依赖 AI 模型，这是传统 12207 未显式覆盖的。参考 15504 评估时对“过程上下文”的适配，加入这两个过程以保证数据和质量的可追溯性。

强化信息管理过程：处理个人健康数据的隐私合规，将安全与隐私活动嵌入需求、设计、测试各阶段，而非仅依赖一个独立的安全过程，体现“剪裁增强”。

简化系统合格性测试：通过用户验收测试直接替代，减少正式文档化测试报告的代价，符合敏捷原则。

生命周期模型的选择：以 Scrum 为骨架，但在前期插入短暂的“架构与数据基线建立”阶段，使敏捷的同时保障基础设计的稳定性，这是对标准生命周期模型管理过程的适应性剪裁。

层次性剪裁实现

依据“软件过程定义的层次性”，先在组织层面定义通用的生命周期模型与质量策略，再在项目层面针对健康膳食系统的特性进行二次剪裁。例如：组织级过程要求每个项目必须有风险管理与配置管理，项目级则细化了风险清单和配置项的范围，舍弃了其余不相关的部分。

通过这样的定义与剪裁，健康膳食系统项目在保持与国际标准一致性的同时，实现了轻量级、且不失严谨的过程执行，保障了项目的成功交付与用户隐私的合规性。

参考文献

ISO/IEC 12207:2017, Systems and software engineering-Software life cycle processes.

ISO/IEC 15504 (series), Information technology-Process assessment.

ISO/IEC 33001:2015, Information technology-Process assessment-Concepts and vocabulary.